La importancia del desarrollo para el buen diseño del software

N L González Morales.[1]

RESUMEN

En este ensayo se examinan los temas vistos en clase que son Desarrollo de Orientado a Objetos y Arquitectura de modelo 4+1 vistas.

El funcionamiento del Desarrollo Orientada a Objetos es todo aquello que tiene características que lo hacen único e indivisible dentro del entorno al que pertenece. Siempre es posible establecer propiedades o atributos de un objeto, lo mismo que su grado de respuesta a estímulos externos (comportamientos del objeto). Sus características y su respuesta ante el entorno. Un elemento notable es que las características de un objeto pueden ser por sí mismas otros objetos.

La importancia de tener un modelo de desarrollo, que sea eficiente y eficaz, hace que se tomen estándares necesarios para suplir todas las necesidades en el proceso de desarrollo de software, el modelo 4+1, es uno de ellos y fue propuesto por el SR Philippe B. Kruchten. Utilizamos la arquitectura del modelo 4+1 para el modelo enfocado en ser claro para la obtención de información del mismo sistema, por medio de los diagramas UML conteniendo los diagramas de casos de uso por ende 4+1 define 4 vistas principales.

Introducción

El software es algo indispensable en nuestra vida diaria que de manera general se entiende como programas de computadora; el desarrollo de software es una actividad donde se involucra la ingeniería; que es el "estudio y aplicación, por especialistas, de las diversas ramas de la tecnología."

Desarrollar un software significa construirlo simplemente mediante su descripción. Está es una muy buena razón para considerar la actividad de desarrollo de software como una ingeniería. Por lo que un producto de software no se limita solo a programas de computadora sino a la documentación asociada a este en las distintas etapas que interviene desde su concepción, análisis, diseño, implementación, pruebas y mantenimiento. El software no estaría completo si no existiera una especificación de requerimiento o un diseño de la arquitectura, crear software es muy similar a la creación y diseño de muchas otras áreas como la arquitectura donde podemos empezar diseñando una casa o departamento hasta un gran rascacielos o el puente más largo que comunica dos ciudades. La ingeniería de software como otras ingeniería hace uso de metodologías que involucran herramientas métodos procedimientos y técnicas para realizar un proyecto por eso es que en este manual se intenta dar una descripción de los pasos que se involucran en el desarrollo de software de acuerdo los requerimientos de los diferentes dominios de su arquitectura.

El desarrollo de sistemas de software no es el simple hecho de programar computadoras, instrucciones en una placa de circuitos integrados para que realice una determinada función o el realizar grandes programas que se ejecuta en una poderosa computadora que puede realizar miles de operaciones por segundo o que de igual manera se puede ejecutar en una PDA o en algún otro dispositivo electrónico; por lo que este proceso involucra a la Ingeniería

[1] N L González Morales alumno del Tecnológico de Estudios Superiores de Ecatepec

de Software que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema (Somerville, 2005, p.6).

Desarrollo

El presente documento intenta dar a conocer y describir los conceptos y aspectos fundamentales del diseño orientado a objetos (DOO) y la arquitectura de 4+1vistas, así como las técnicas, metodologías y herramientas actuales de dicho paradigma en la Ingeniería de software.

Así pues, definimos el **Diseño Orientado a los Objetos (DOO)** crea una representación del problema del mundo real y la hace corresponder con el ámbito de la solución, que es el software.

A diferencia con otros métodos de diseño, el DOO produce un diseño que interconecta objetos de datos y operaciones de procesamiento para esos objetos, de forma que se modulariza la información y el procesamiento, en lugar de aislar el procesamiento.

Todos los métodos de diseño intentan desarrollar software basándose en:

• Abstracción

• Ocultamiento de información

• Modularidad

El DOO proporciona un mecanismo que permite al diseñador consigue estas tres características sin dificultad.

El Análisis Orientado a Objetos, el Diseño Orientado a Objetos y la Programación Orientada a Objetos comprenden un conjunto de actividades de la Ingeniería del Software para la construcción de un sistema basado en objetos.

El Diseño Orientado a Objetos se define como un diseño de sistemas que utiliza objetos auto-contenidos y clases de objetos.

En el modelo 4+1 surge esta necesidad cuando un desarrollador inicia el modelado de un problema; lógicamente en este proceso se hace énfasis en proporcionar la mayor información del mismo a través de los diagramas que se utilicen para su descripción, esto trae consigo que puedan suceder conflictos en la representación de la arquitectura del sistema. Esta arquitectura de software propuesta por Kruchten es una forma de resolver esta problemática.

El modelo describe la arquitectura de software del sistema a través de 5 vistas concurrentes Kruchten las agrupa estas 5 vistas por su naturaleza en 3 apartados; el conceptual donde sitúa a la vista lógica y la de procesos, el físico que lo componen las vistas de componentes y la distribuida y por último la funcional la que se refiere a la vista de casos de uso.

- Vista Lógica
- Vista Concurrente o de Procesos
- Vista de Componentes o de desarrollo
- Vista distribuida o física
- Vista de casos de uso de escenarios

¿Qué es el Desarrollo Orientado a Objetos? Este proporciona un enfoque que se considera bastante adecuado para construir sistemas que respondan rápidamente a los entornos cambiantes de negocios, incluyendo aplicaciones web. El desarrollo Orientado a objetos considera al objeto como la unidad básica del análisis y diseño de sistemas. El sistema se modela como una colección de objetos y de las relaciones que existen entre ellos. [2]

Las fases del desarrollo Orientado a objetos son similares a las desarrollo de los sistemas convencionales, que constan del análisis, diseño e implementación. Sin embargo el desarrollo es más iterativo e incremental que el desarrollo estructurado tradicional. Durante el análisis, los constructores de sistemas documentan los requerimientos funcionales del sistema, especificando sus propiedades más importantes y lo que el sistema propuesto debe hacer. Se analizan las interacciones entre el sistema y los usuarios para identificar objetos, los cuales incluyen tanto datos como procesos.

Las fases del diseño Orientado a Objetos describen como se comportaran los objetos y como interactuaran unos con otros. Los objetos similares se agrupan para formar una clase y las clases en jerarquías, en las que una subclase hereda los atributos y métodos de la superclase.

Al igual que todos los métodos de diseño utilizan su propia notación y metodología, el DOO introduce un conjunto nuevo de términos, notaciones y procedimientos para la derivación del diseño del software.

Se resume la terminología orientada a objetos e introducimos algunos conceptos propios de esta forma de diseño:

Para conseguir un DOO, tenemos que establecer un mecanismo para:

- Representar la estructura de datos
- Especificar el proceso
- Realizar el procedimiento de invocación

Objeto: Componente del mundo real que se hace corresponder con el software. En un Sistema de Información basado en Computador, un objeto es un producto o consumidor de información, o un elemento de información.

Mensajes: Peticiones que se realizan a los objetos para que realicen alguna de sus operaciones. Las operaciones contienen construcciones procedimentales y de control, que se

[2] (Grau, s.f.)

invocan mediante un mensaje.

Pasos del DOO

Esencialmente, el DOO consta de cuatro pasos fundamentales:

- Identificación y definición de objetos y clases
- Organización de relaciones entre clases
- Extracción de estructuras en una jerarquía de clases
- Construcción de bibliotecas de clases reutilizables

Como en todas las fases de Ingeniería del Software, el DOO es cíclico, es decir, los diseñadores comienzan definiendo un conjunto de clases, que se amplían, modifican, y vuelta a empezar.

Identificación y definición de objetos

El principal problema del desarrollo de un sistema orientado a objetos es encontrar los objetos en la fase de AOO y DOO.

El método que utilizaremos para la identificación de objetos es el propuesto por Booch en 1983, que dio origen al método gramatical.

Esta metodología sugiere que se comience con una descripción textual del sistema deseado y que el diseñador vea:

- A los nombres como posibles identificadores de las clases de los objetos
- A los verbos como posibles métodos
- Si se representara como un método de otra clase, pocos usuarios de ésta lo invocarían.

.

MODELO 4+1 VISTAS

Philippe Krutchten

Este artículo presenta un modelo para describir la arquitectura de sistemas de sistemas de software, basándose en el uso de múltiples vistas concurrente. Este uso de múltiples vistas permite abordar los intereses de los distintos "stakeholders" de la arquitectura por separado: usuarios finales, desarrolladores ingenieros de sistemas, administradores de proyecto, etc. Y manejar los requisitos funcionales y no funcionales separadamente. Se describe cada una de las cinco distintas vistas descritas, conjuntamente con la notación para captarla. Las vistas se diseñan mediante un proceso centrado en la arquitectura, motivado por escenarios y desarrollado iterativamente. [3]

La arquitectura del software se trata de abstracciones, de descomposición, y composición, de estilos y estética. También tiene relación con el diseño y la implementación de la estructura de alto nivel del software.

Estas vistas se presenta tradicionalmente en una figura de cuatro cajas con un ovalo central que representa al modelo de casos de uso.

[3] (Object-Oriented Analysis and Design with Applications., 1993)

Los diseñadores construyen la arquitectura usando varios elementos arquitectónicos elegidos apropiadamente. Estos elementos satisfacen la mayor parte de los requisitos de funcionalidad y performance del sistema, así como también otros requisitos no funcionales tales como confiablidad, escabilidad, portabilidad y disponibilidad del sistema.

- **Vista de Casos de Uso**: que contiene requisitos desarrollados en las restantes vistas.

- **Vista Lógica**: La arquitectura lógica apoya principalmente los requisitos funcionales –lo que el sistema debe brindar en términos de servicios a sus usuarios. El sistema se descompone en una serie de abstracciones clave, tomadas (principalmente) del dominio del problema en la forma de objetos o clases de objetos. Aquí se aplican los principios de abstracción, encapsulamiento y herencia. Esta descomposición no solo se hace para potenciar el análisis funcional, sino también sirve para idéntica mecanismos y elementos de diseños comunes a diversas partes del sistema.

- **Vista Física**: La arquitectura física toma en cuenta primeramente los requisitos no funcionales del sistema tales como la disponibilidad, confiabilidad (tolerancia a fallas), performance (throughput), y escalabilidad. El software ejecuta sobre una red de computadores o nodos de procesamiento (o tan solo nodos). Los variados elementos identificados redes, procesos, tareas y objetos requieren ser mapeados sobre los variados nodos. Esperamos que diferentes configuraciones puedan usarse: algunas para desarrollo y pruebas, otras para emplazar el sistema en varios sitios para distintos usuarios.

- **Vista de Procesos**: La arquitectura de procesos toma en cuenta algunos requisitos no funcionales tales como la performance y la disponibilidad. Se enfoca en asuntos de concurrencia y distribución, integridad del sistema, de tolerancia a fallas. La vista de procesos también especiada en cual hilo de control se ejecuta efectivamente una operación de una clase identificada en la vista lógica.

- **Vista de Desarrollo**: La vista de desarrollo se centra en la organización real de los módulos de software en el ambiente de desarrollo del software. El software se empaqueta en partes pequeñas bibliotecas de programas o subsistemas que pueden ser desarrollados por uno o un grupo pequeños desarrolladores. Los subsistemas se organizan en una jerarquía de capas, cada una de las cuales brinda una interfaz estrecha y bien definida hacia las capas superiores.

Usamos dos estrategias concurrentemente para determinar la cantidad correcta de concurrencia y definir el conjunto de procesos que se necesitan. Considerando el conjunto de posibles arquitecturas físicas, podemos proceder o bien:

- Inside-out: Comenzando a partir de la arquitectura lógica: definir las tareas agentes que multiplexan un único thread de control entre múltiples objetos activos de una clase; los objetos cuya persistencia o vida está subordinada a un objeto activo también se ejecutan en ese mismo agente; muchas clases que requieren ser ejecutadas con mutua exclusión, o que requieren solo un pequeño procesamiento comparten el mismo agente. Este clustering prosigue hasta que se reducen los procesos hasta un número razonablemente pequeño que aún permite distribución y uso de los recursos físicos.
- Outside-in: Comenzando con la arquitectura física: idéntica los estímulos externos (requerimientos) al sistema, definir los procesos cliente para manejar los estímulos y procesos servidores que sólo brindan servicios y que no los inician; usar la integridad de los datos y las restricciones de serializacion del problema para definir el conjunto correcto de servidores, y asignar objetos a los agentes cliente y servidor; idéntica cuales objetos deben ser distribuidos.

Los elementos de las cuatro vistas trabajan conjuntamente en forma natural mediante el uso de un conjunto pequeño de escenarios relevantes –instancias de casos de uso más generales– para los cuales describimos sus scripts correspondientes (secuencias de interacciones entre objetos y entre procesos) tal como lo describen Rubin y Goldberg [10]. Los escenarios son de alguna manera una abstracción de los requisitos más importantes.

Su diseño se expresa mediante el uso de diagramas de escenarios y diagramas de interacción de objetos [3]. Esta vista es redundante con las otras (y por lo tanto “+1”), pero sirve a dos propósitos principales:

Como una guía para descubrir elementos arquitectónicos durante el diseño de arquitectura tal como lo describiremos más adelante

Como un rol de validación e ilustración después de completar el diseño de arquitectura, en el papel y como punto de partido de las pruebas de un prototipo de la arquitectura.

Metodología

¿Por qué es de vital importancia conocer estos temas?

En la arquitectura 4+1vistas, la importancia de conocer este modelo es que a la hora de desarrollarlo se hace énfasis en proporcionar una mayor información del mismo a través de diagramas que nos ayuden a tener una mejor descripción. De esta manera la finalidad de juntar estas 5 vistas es poder tener una mejor comunicación entre desarrolladores para tener una mejor comprensión del diseño.

Sin embargo el Desarrollo Orientado a Objetos nos es de gran utilidad al igual que el modelo anterior, pero a diferencia que este desarrollamos una Planeación y Especificación de Requisitos, Construcción (Análisis, Diseño e Implementación) e Instalación. Con esta aproximación se consigue disminuir el grado de complejidad para tener de esta forma un sistema funcionando ayudara a la comunicación entre el usuario/cliente.

Conclusiones

Este desarrollo proporciona un enfoque que se considera bastante adecuado para construir diseñar e implementar software que pueda facilitar a los negocios como aplicaciones web.

En el modelo 4+1, la finalidad la necesidad de facilitar la comunicación entre los diseñadores y usuarios para establecer una mayor comprensión del sistema que se esté modelando.

Referencias

K.P. Birman and R. Van Renesse, Reliable Distributed Computing with the Isis Toolkit, IEEE CS Press, and Los Alamitos, Calif. 1994.

4+1 View Model of Software Architecture IEEE Software. Kruchtn P. Architectural Blueprints,November 1995

Diseño Orientado a Objetos http://indalog.ual.es/mtorres/LP/DOO.pdf

Desarrollo de Proyectos de Software http://www.tesoem.edu.mx/alumnos/cuadernillos/2011.009.pdf

Planos Arquitectónicos: El Modelo de “4+1” Vistas de la Arquitectura del Software∗ http://cic.puj.edu.co/wiki/lib/exe/fetch.php?media=materias:modelo4_1.pdf